JUC 并发编程

零、思维导图

一、

二、

三、

四、

五、

六、

七、

八、

九、

十、

十一、

十二、

十三、

十四

十五、

十六、

十七、

十八、

十九、

二十、

1、

2、

3、

4、

5、

6、

7、

8、

9、

①、

②、

③、

④、

⑤、

⑥、

⑦、

⑧、

⑨、

⑩、

⑪、⑫、⑬、⑭、⑮、⑯、⑰、⑱、⑲、⑳

㉑、㉒、㉓、㉔、㉕、㉖、㉗、㉘、㉙、㉚

㉛、㉜、㉝、㉞、㉟、㊱、㊲、㊳、㊴、㊵

㊶、㊷、㊸、㊹、㊺、㊻、㊼、㊽、㊾、㊿

Ⅰ、

Ⅱ、

Ⅲ、

Ⅳ、

Ⅴ、

Ⅵ、

Ⅶ、

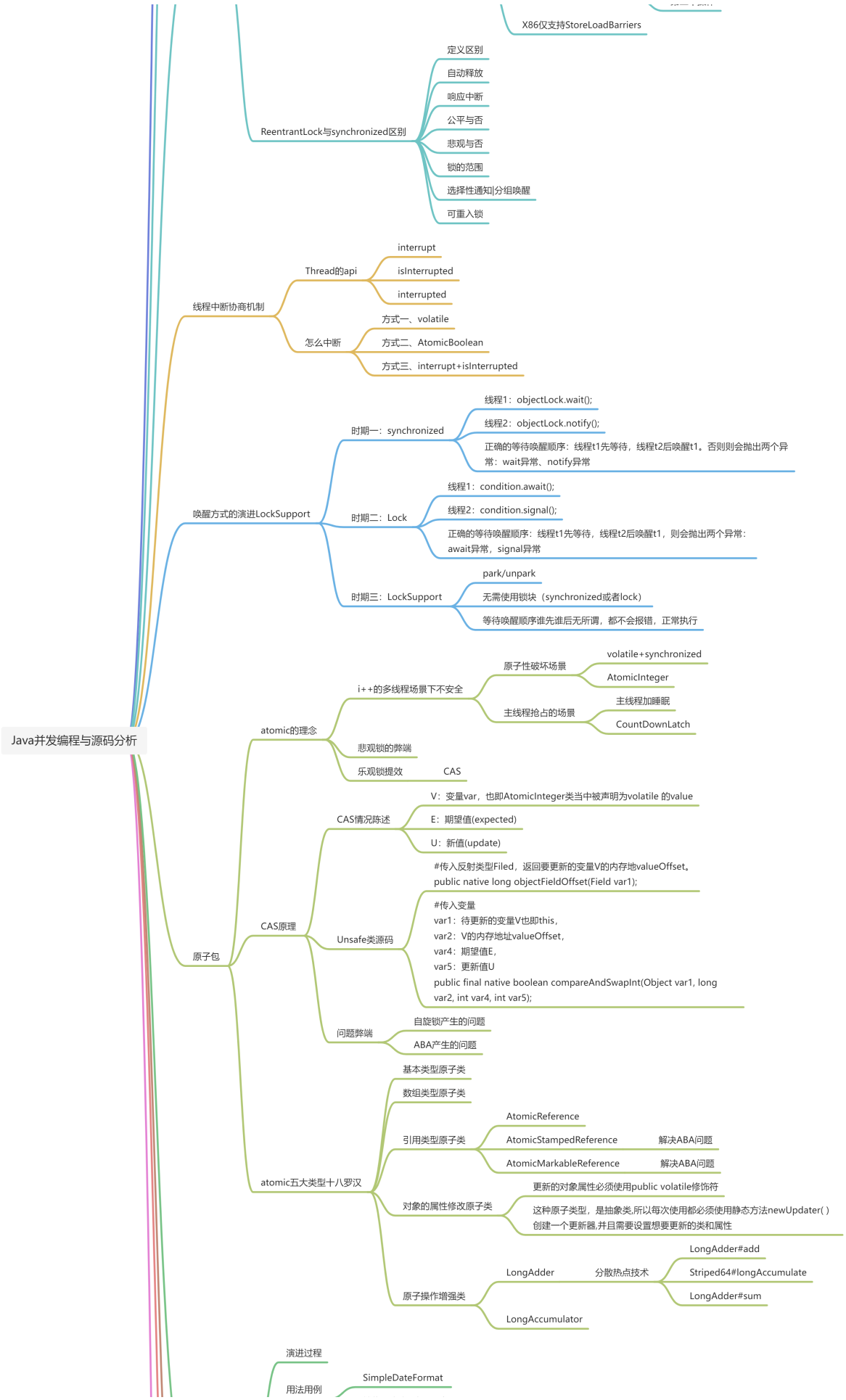
Ⅷ、

Ⅸ、

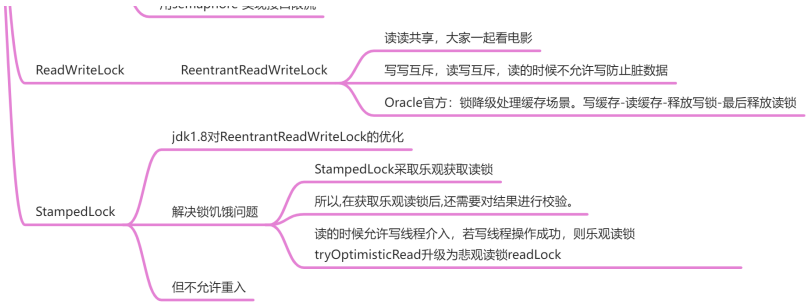
Ⅹ、

零、思维导图

- 线程基础概念
  - 简单的start
    - 涉及操作系统源码
      - os::start_thread(thread)
        - openjdk8\hotspot\src\share\vm\prims　　thread.cpp
      - 证明多线程是操作系统范畴，与语言无关
  - 多线程相关概念
    - 1把锁　　synchronized
    - 2个并
      - 并发concurrent（抢票、秒杀）
        - 是在同一实体上的多个事件
        - 是在一台处理器上同时处理多个任务　　同一时刻，其实是只有一个事件发生
      - 并行parallel（泡泡面）
        - 是在不同实体上的多个事件
        - 是在多台处理器上同时处理多个任务　　同一时刻，大家真的都在做事情，你做你的，我做我的
    - 3个程
      - 进程：系统当中运行的一个个应用程序
      - 线程：轻量级进程，同一个进程内会有1个或多个线程，线程也是操作系统进行调度的基本单元
      - 管程：Monitor即监视器，也即是我们平时说的锁
  - 线程分类
    - 用户线程（User Thread）
      - 主线程
      - 用户创建的线程
      - 不同线程的生命周期不同，一般不会同时消亡
    - 守护线程 （Daemon Thread）
      - t1.setDaemon(true);
      - gc线程
      - 用户线程全部消亡则守护线程也消亡
  - 问题产生
    - i++多线程不安全
    - 集合不安全
    - 线程锁怎么用
- Future接口的异步任务演进
  - 异步任务即开辟分支任务，不阻塞主线程。Tips：异步线程的创建是纳秒级别
  - FutureTask
    - 三合一功能
      - 作为线程：实现了Runnable接口
      - 异步处理：实现了Future接口
      - 有返回值：构造注入了Callable<T>，提供了Callable功能
    - 配合线程池
      - 避免手动多次创建线程，代码清爽
      - 线程复用，节约资源
    - FutureTask缺点
      - 阻塞方式get
        - futureTask.get()：get方法会阻塞主线程，一般要把get方法放到最后
        - futureTask.get(3,TimeUnit.SECONDS)：假如我不愿意等待很长时间，过时不候直接抛出TimeOutException。其他程序调用的时候可以捕获超时异常，相当于变相的做了止损操作
      - 轮询方式isDone
        - 不断给主线程续命以等待异步线程的返回
        - 徒劳消耗cpu资源
  - CompletableFuture
    - 创建方式
      - runAsync
        - runAsync(Runnable runnable)
        - runAsync(Runnable runnable,Executor executor)
      - supplyAsync
        - supplyAsync(Supplier<U> supplier)
        - supplyAsync(Supplier<U> supplier,Executor executor)
      - 是否指定线程池
        - 指定线程池Executor 的时候，要记得在finally中关闭线程池
        - 若不指定线程池，则CompletableFuture使用了默认的线程池ForkJoinPool，主线程切忌立马结束，否则默认线程池就会立刻关闭，最终导致分支线程失效。此时ForkJoinPool充当了守护线程
    - CompletableFuture优点
      - 减少阻塞：实现了回调接口CompletionStage
        - whenComplete
        - exceptionally
      - 无需轮询
    - 取值方法对比
      - get：编译期检查，需要抛异常
      - join：运行时才检查，无需抛异常
    - 多任务组合处理
      - 线程串行化
      - 双线程均完成才能后续
      - 双线程完成其一就能后续
      - 多任务组合
- 本地锁
  - synchronized
  - volatile
    - 保证可见性
    - 不保证原子性
    - 保证有序性
      - happen-before理念
      - 禁止代码重排序
        - 四大内存屏障指令
          - LoadLoadBarriers
          - StoreStoreBarriers
          - LoadStoreBarriers
          - StoreLoadBarriers**最强大花销也最大，Buffer Fully Flush
        - 编译器制定的重排序规则表
          - 第一个操作
          - 第二个操作


Java并发编程与源码分析
X86仅支持StoreLoadBarriers
ReentrantLock与synchronized区别
定义区别
自动释放
响应中断
公平与否
悲观与否
锁的范围
选择性通知|分组唤醒
可重入锁
线程中断协商机制
Thread的api
interrupt
isInterrupted
interrupted
怎么中断
方式一、volatile
方式二、AtomicBoolean
方式三、interrupt+isInterrupted
唤醒方式的演进LockSupport
时期一：synchronized
线程1：objectLock.wait();
线程2：objectLock.notify();
正确的等待唤醒顺序：线程t1先等待，线程t2后唤醒t1。否则则会抛出两个异常：wait异常、notify异常
时期二：Lock
线程1：condition.await();
线程2：condition.signal();
正确的等待唤醒顺序：线程t1先等待，线程t2后唤醒t1，则会抛出两个异常：await异常，signal异常
时期三：LockSupport
park/unpark
无需使用锁块（synchronized或者lock）
等待唤醒顺序谁先谁后无所谓，都不会报错，正常执行
原子包
atomic的理念
i++的多线程场景下不安全
原子性破坏场景
volatile+synchronized
AtomicInteger
主线程抢占的场景
主线程加睡眠
CountDownLatch
悲观锁的弊端
乐观锁提效
CAS
CAS原理
CAS情况陈述
V：变量var，也即AtomicInteger类当中被声明为volatile 的value
E：期望值(expected)
U：新值(update)
Unsafe类源码
#传入反射类型Filed，返回要更新的变量V的内存地valueOffset。
public native long objectFieldOffset(Field var1);
#传入变量
var1：待更新的变量V也即this，
var2：V的内存地址valueOffset，
var4：期望值E，
var5：更新值U
public final native boolean compareAndSwapInt(Object var1, long var2, int var4, int var5);
问题弊端
自旋锁产生的问题
ABA产生的问题
atomic五大类型十八罗汉
基本类型原子类
数组类型原子类
引用类型原子类
AtomicReference
AtomicStampedReference
解决ABA问题
AtomicMarkableReference
解决ABA问题
对象的属性修改原子类
更新的对象属性必须使用public volatile修饰符
这种原子类型，是抽象类,所以每次使用都必须使用静态方法newUpdater( )创建一个更新器,并且需要设置想要更新的类和属性
原子操作增强类
LongAdder
分散热点技术
LongAdder#add
Striped64#longAccumulate
LongAdder#sum
LongAccumulator
演进过程
用法用例
SimpleDateFormat

- ThreadLocal
  - 销售员卖房子|CS玩家
  - 源码
    - Thread
    - ThreadLocal
  - 线程池场景下的线程经常会被复用
    - 影响后续业务逻辑
    - 和造成内存泄露等问题
  - 四大引用扩展
    - 强引用
    - 软引用
    - 弱引用
    - 虚引用
- 对象内存布局
  - 对象头
    - markword
    - 类型指针
  - 实例数据
  - 数据对齐
- Synchronized锁升级
  - markword占8个字节即64位
  - 无锁
  - 偏向锁
    - Java6
    - Java15废除
  - 轻量锁 自旋CAS
  - 重量级锁
    - Java5
    - synchronized的底层原理，对象监视器 monitor
      - monitorenter /monitorexit
      - ACC_SYNCHRONIZED
    - 操作系统
      - 用户态
      - 内核态
      - mutexLock指令
  - JIT语法优化
    - 锁消除
    - 锁粗化
  - 小口诀
    - 无锁：自由自在
    - 偏向锁：唯我独尊
    - 轻量锁：楚汉争霸
    - 重量级锁：群雄逐鹿
- Locks锁包
  - AbstractQueuedSynchronizer原理
    - 数据结构
      - 循环双向线程阻塞队列节点Node
        - Node类型
          - SHARED
          - EXCLUSIVE
        - waitStatus状态
          - CANCELLED = 1;
          - SIGNAL = -1;
          - CONDITION = -2;
          - PROPAGATE = -3;
        - volatile int waitStatus; 默认为0
        - volatile Thread thread;
        - volatile Node prev;
        - volatile Node next;
      - volatile int state; 0-1-2-3....
    - 非公平锁与公平锁原理
      - NonfairSync
      - FairSync
    - 尝试获取锁
      - tryAcquire
      - addWaiter
      - acquireQueued
    - 唤醒阻塞线程
      - unlock release | tryRelease | unparkSuccessor(h)【LockSupport.unpark】
      - acquireQueued
  - Lock ReentrantLock
  - CountDownLatch
    - 利用枚举减少if else的判断
    - CountDownLatch去解决时间等待问题
  - CyclicBarrier
    - CyclicBarrier阻塞任务线程而非主线程 集齐7颗龙珠就能召唤神龙
    - 兄弟四人打篮球
      - 屏障的共同抵达提前执行操作，构造方法传入Runnable实例
      - 线程具体的业务操作
    - 屏障复用 又来了一拨人，看看愿不愿意一起打：
  - Semaphore
    - 用semaphore 实现停车场提示牌功能
    - 用semaphore 实现防止商品超卖
    - 用semaphore 实现接口限流


ReadWriteLock
ReentrantReadWriteLock
读读共享，大家一起看电影
写写互斥，读写互斥，读的时候不允许写防止脏数据
Oracle官方：锁降级处理缓存场景。写缓存-读缓存-释放写锁-最后释放读锁
StampedLock
jdk1.8对ReentrantReadWriteLock的优化
解决锁饥饿问题
StampedLock采取乐观获取读锁
所以,在获取乐观读锁后,还需要对结果进行校验。
读的时候允许写线程介入，若写线程操作成功，则乐观读锁
tryOptimisticRead升级为悲观读锁readLock
但不允许重入

一、

二、

三、

四、

五、

六、

七、

八、

九、

十、

十一、

十二、

十三、

十四

十五、

十六、

十七、

十八、

十九、

二十、

1、

2、

3、

4、

5、

6、

7、

8、

9、

---

①、

②、

③、

④、

⑤、

⑥、

⑦、

⑧、

⑨、

⑩、

⑪、⑫、⑬、⑭、⑮、⑯、⑰、⑱、⑲、⑳

㉑、㉒、㉓、㉔、㉕、㉖、㉗、㉘、㉙、㉚

㉛、㉜、㉝、㉞、㉟、㊱、㊲、㊳、㊴、㊵

㊶、㊷、㊸、㊹、㊺、㊻、㊼、㊽、㊾、㊿

Ⅰ、

Ⅱ、

Ⅲ、

Ⅳ、

Ⅴ、

Ⅵ、

Ⅶ、

Ⅷ、

Ⅸ、

Ⅹ、